```
import numpy as np
import pandas as pd

CustomerData = pd.read_excel("/content/CustomersData.xlsx")
Discount_Coupon = pd.read_csv("/content/Discount_Coupon.csv")
Marketing_Spend = pd.read_csv("/content/Marketing_Spend.csv")
Online_Sales = pd.read_csv("/content/Online_Sales.csv")
Tax_Amount = pd.read_excel("/content/Tax_amount.xlsx")
```

```
#Load and Inspect Data:Shape
print(CustomerData.shape)
print(Discount_Coupon.shape)
print(Marketing_Spend.shape)
print(Online_Sales.shape)
print(Tax_Amount.shape)
```

```
(1468, 4)
(204, 4)
(365, 3)
(52924, 10)
(20, 2)
```

```
#Load and Inspect Data:Preview data
CustomerData.head()
Discount_Coupon.head()
Marketing_Spend.head()
Online_Sales.head()
Tax_Amount.head()
```

| | Product_Category | GST |
|---|---|---|
| 0 | Nest-USA | 0.10 |
| 1 | Office | 0.10 |
| 2 | Apparel | 0.18 |
| 3 | Bags | 0.18 |
| 4 | Drinkware | 0.18 |

Next steps: Generate code with Tax_Amount | New interactive sheet

```
#Load and Inspect Data:Understand structure & data types
CustomerData.info()
Discount_Coupon.info()
Marketing_Spend.info()
Online_Sales.info()
Tax_Amount.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1468 entries, 0 to 1467
Data columns (total 4 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   CustomerID     1468 non-null   int64
 1   Gender         1468 non-null   object
 2   Location       1468 non-null   object
 3   Tenure_Months  1468 non-null   int64
dtypes: int64(2), object(2)
memory usage: 46.0+ KB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 204 entries, 0 to 203
Data columns (total 4 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Month             204 non-null    object
 1   Product_Category  204 non-null    object
 2   Coupon_Code       204 non-null    object
 3   Discount_pct      204 non-null    int64
dtypes: int64(1), object(3)
memory usage: 6.5+ KB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 365 entries, 0 to 364
```

```
Data columns (total 5 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Date           365 non-null    object
 1   Offline_Spend  365 non-null    int64
 2   Online_Spend   365 non-null    float64
dtypes: float64(1), int64(1), object(1)
memory usage: 8.7+ KB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52924 entries, 0 to 52923
Data columns (total 10 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   CustomerID           52924 non-null  int64
 1   Transaction_ID       52924 non-null  int64
 2   Transaction_Date     52924 non-null  object
 3   Product_SKU          52924 non-null  object
 4   Product_Description  52924 non-null  object
 5   Product_Category     52924 non-null  object
 6   Quantity             52924 non-null  int64
 7   Avg_Price            52924 non-null  float64
 8   Delivery_Charges     52924 non-null  float64
 9   Coupon_Status        52924 non-null  object
dtypes: float64(2), int64(3), object(5)
memory usage: 4.0+ MB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 2 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Product_Category  20 non-null     object
 1   GST               20 non-null     float64
dtypes: float64(1), object(1)
memory usage: 452.0+ bytes
```

```
#Load and Inspect Data:Check missing values
#Decide how to handle missing values
print(CustomerData.isnull().sum())
print(Discount_Coupon.isnull().sum())
print(Marketing_Spend.isnull().sum())
print(Online_Sales.isnull().sum())
print(Tax_Amount.isnull().sum())
```

```
CustomerID       0
Gender           0
Location         0
Tenure_Months    0
dtype: int64
Month               0
Product_Category    0
Coupon_Code         0
Discount_pct        0
dtype: int64
Date             0
Offline_Spend    0
Online_Spend     0
dtype: int64
CustomerID             0
Transaction_ID         0
Transaction_Date       0
Product_SKU            0
Product_Description    0
Product_Category       0
Quantity               0
Avg_Price              0
Delivery_Charges       0
Coupon_Status          0
dtype: int64
Product_Category    0
GST                 0
dtype: int64
```

```
#Load and Inspect Data:Fix common data-type issues
# IDs → strings
# Labels → category
# Time → datetime
# Amounts → numeric

# CustomerData
CustomerData["CustomerID"] = CustomerData["CustomerID"].astype(str)
```

```
CustomerData["Gender"] = CustomerData["Gender"].astype("category")
CustomerData["Location"] = CustomerData["Location"].astype("category")

# Discount_Coupon
Discount_Coupon["Month"] = Discount_Coupon["Month"].astype("category")
Discount_Coupon["Product_Category"] = Discount_Coupon["Product_Category"].astype("category")
Discount_Coupon["Coupon_Code"] = Discount_Coupon["Coupon_Code"].astype(str)

# Marketing_Spend
Marketing_Spend["Date"] = pd.to_datetime(
    Marketing_Spend["Date"],
    format="%m/%d/%Y",
    errors="coerce"
)

# Online_Sales
Online_Sales["CustomerID"] = Online_Sales["CustomerID"].astype(str)
Online_Sales["Transaction_ID"] = Online_Sales["Transaction_ID"].astype(str)
Online_Sales["Transaction_Date"] = pd.to_datetime(
    Online_Sales["Transaction_Date"],
    format="%m/%d/%Y",
    errors="coerce"
)
Online_Sales["Product_Category"] = Online_Sales["Product_Category"].astype("category")
Online_Sales["Coupon_Status"] = Online_Sales["Coupon_Status"].astype("category")

# Tax_Amount
Tax_Amount["Product_Category"] = Tax_Amount["Product_Category"].astype("category")
```

```
CustomerData.info()
Discount_Coupon.info()
Marketing_Spend.info()
Online_Sales.info()
Tax_Amount.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1468 entries, 0 to 1467
Data columns (total 4 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   CustomerID     1468 non-null   object
 1   Gender         1468 non-null   category
 2   Location       1468 non-null   category
 3   Tenure_Months  1468 non-null   int64
dtypes: category(2), int64(1), object(1)
memory usage: 26.3+ KB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 204 entries, 0 to 203
Data columns (total 4 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Month             204 non-null    category
 1   Product_Category  204 non-null    category
 2   Coupon_Code       204 non-null    object
 3   Discount_pct      204 non-null    int64
dtypes: category(2), int64(1), object(1)
memory usage: 4.8+ KB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 365 entries, 0 to 364
Data columns (total 3 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Date           365 non-null    datetime64[ns]
 1   Offline_Spend  365 non-null    int64
 2   Online_Spend   365 non-null    float64
dtypes: datetime64[ns](1), float64(1), int64(1)
memory usage: 8.7 KB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52924 entries, 0 to 52923
Data columns (total 10 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   CustomerID           52924 non-null  object
 1   Transaction_ID       52924 non-null  object
 2   Transaction_Date     52924 non-null  datetime64[ns]
 3   Product_SKU          52924 non-null  object
 4   Product_Description  52924 non-null  object
 5   Product_Category     52924 non-null  category
```

```
 6   Quantity            52924 non-null  int64
 7   Avg_Price           52924 non-null  float64
 8   Delivery_Charges    52924 non-null  float64
 9   Coupon_Status       52924 non-null  category
dtypes: category(2), datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 3.3+ MB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 2 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Product_Category  20 non-null     category
 1   GST               20 non-null     float64
dtypes: category(1), float64(1)
memory usage: 1.0 KB
```

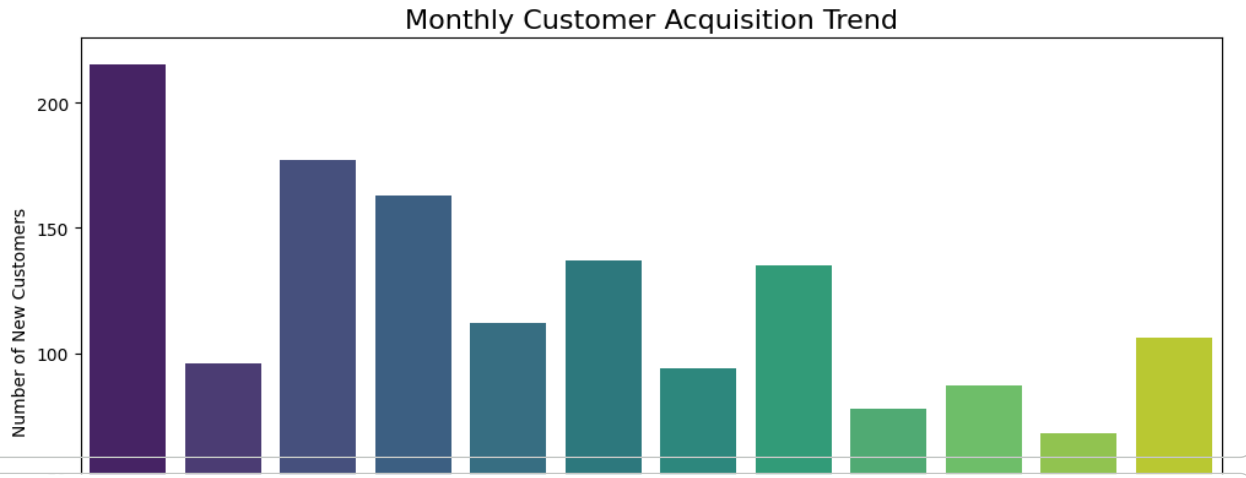
```
#Calculate Core Metrics:
#Invoice Value = ((Quantity * Avg_price) * (1 - Discount_pct) * (1 + GST)) + Delivery_Charges.
Discount_Coupon['Discount_pct'] = Discount_Coupon['Discount_pct'] / 100

Online_Sales = pd.merge(
    Online_Sales,
    Discount_Coupon[['Product_Category', 'Discount_pct']],
    on='Product_Category',
    how='left'
)

Online_Sales = pd.merge(
    Online_Sales,
    Tax_Amount[['Product_Category', 'GST']],
    on='Product_Category',
    how='left'
)

Online_Sales['Invoice_Value'] = (
    (Online_Sales['Quantity'] * Online_Sales['Avg_Price']) *
    (1 - Online_Sales['Discount_pct']) *
    (1 + Online_Sales['GST'])
) + Online_Sales['Delivery_Charges']

Online_Sales.head()
```

| | CustomerID | Transaction_ID | Transaction_Date | Product_SKU | Product_Description | Product_Category |
|---|---|---|---|---|---|---|
| **0** | 17850 | 16679 | 2019-01-01 | GGOENEBJ079499 | Nest Learning Thermostat 3rd Gen-USA - Stainle... | Nest-USA |
| **1** | 17850 | 16679 | 2019-01-01 | GGOENEBJ079499 | Nest Learning Thermostat 3rd Gen-USA - Stainle... | Nest-USA |
| **2** | 17850 | 16679 | 2019-01-01 | GGOENEBJ079499 | Nest Learning Thermostat 3rd Gen-USA - Stainle... | Nest-USA |
| **3** | 17850 | 16679 | 2019-01-01 | GGOENEBJ079499 | Nest Learning Thermostat 3rd Gen-USA - Stainle... | Nest-USA |
| **4** | 17850 | 16679 | 2019-01-01 | GGOENEBJ079499 | Nest Learning Thermostat 3rd Gen-USA - Stainle... | Nest-USA |

```
# 1. Identify the months with the highest and lowest customer acquisition count. What strategies could
#ensure consistent growth throughout the year?
# Hint: Think about what “acquisition” really means – it’s the first time a customer makes a purchase.
#unique customer using appropriate grouping and aggregation methods. This will help you isolate the fi
# Hint: Once you’ve identified each customer's first transaction date, extract the month from that dat
#and count how many customers fall into each group. This will give you the monthly trend of customer a

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 1: Get first purchase per customer
first_purchase = Online_Sales.groupby('CustomerID')['Transaction_Date'].min().reset_index()
```

```
first_purchase = first_purchase.rename(columns={'Transaction_Date':'First_Purchase_Date'})
first_purchase = first_purchase.set_index('CustomerID')

# Step 2: Extract month name
first_purchase = first_purchase.assign(Acquisition_Month = first_purchase['First_Purchase_Date'].dt.mo

# Step 3: Count customers acquired per month
monthly_acquisition = first_purchase.groupby('Acquisition_Month').size().reset_index(name='Customer_Co

# Step 4: Sort months in calendar order
month_order = ['January','February','March','April','May','June','July','August','September','October'
monthly_acquisition['Acquisition_Month'] = pd.Categorical(
    monthly_acquisition['Acquisition_Month'],
    categories=month_order,
    ordered=True
)
monthly_acquisition = monthly_acquisition.sort_values('Acquisition_Month')

# Step 5: Identify highest and lowest acquisition months
highest_month = monthly_acquisition.loc[monthly_acquisition['Customer_Count'].idxmax()]
lowest_month = monthly_acquisition.loc[monthly_acquisition['Customer_Count'].idxmin()]

print("Highest acquisition month:\n", highest_month)
print("Lowest acquisition month:\n", lowest_month)

#Step 6: Visualize monthly acquisition trend
plt.figure(figsize=(12,6))
sns.barplot(
    data=monthly_acquisition,
    x='Acquisition_Month',
    y='Customer_Count',
    palette='viridis'
)
plt.title("Monthly Customer Acquisition Trend", fontsize=16)
plt.xticks(rotation=45)
plt.ylabel("Number of New Customers")
plt.show()

Online_Sales.head(5)
```

```
Highest acquisition month:
 Acquisition_Month    January
Customer_Count           215
Name: 4, dtype: object
Lowest acquisition month:
 Acquisition_Month    November
Customer_Count             68
Name: 9, dtype: object
/tmp/ipython-input-3100871479.py:41: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `

  sns.barplot(
```



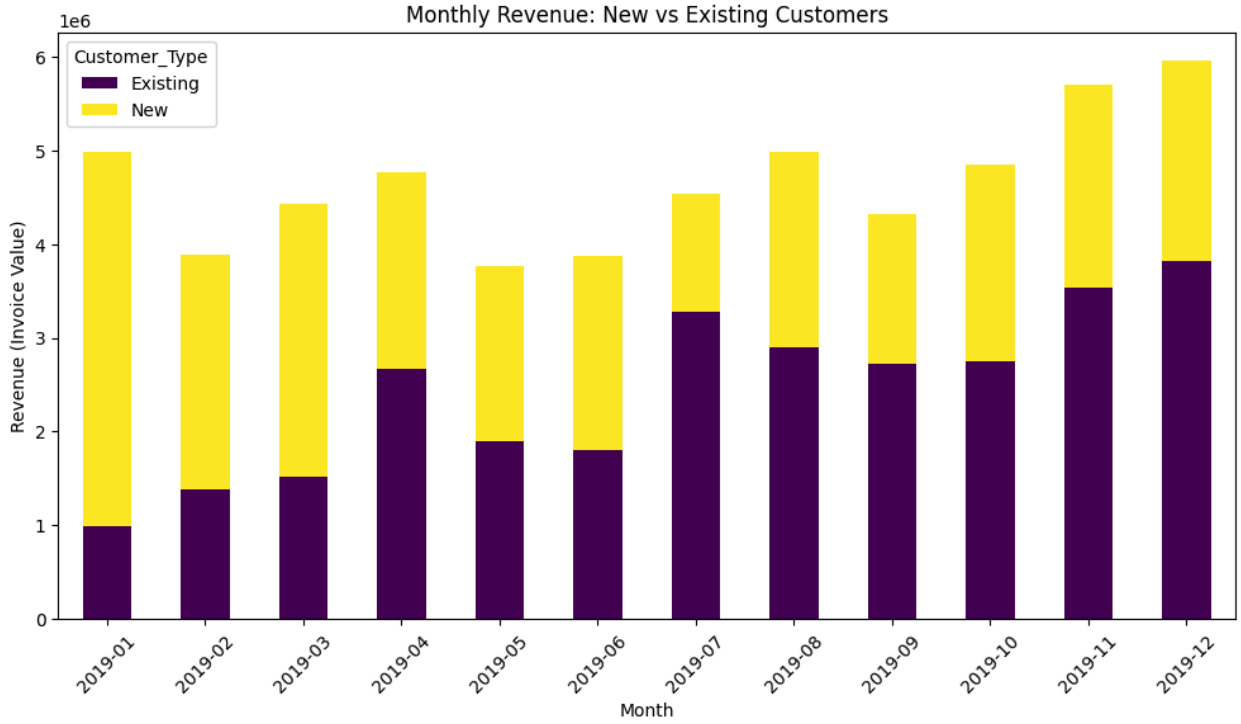

```
# 2. Analyze customer behavior during high-retention months and suggest ways to replicate this success
# Hint: For periods identified with strong retention, delve into the characteristics of purchases made
#applications, or transaction values were common?
# Hint: Think about how these successful behaviors or conditions could be fostered in other months.

# # Monthly Retention rate = (Customers in Month X who returned in Month X+1) /  (total number of cust
# #Retention = customers coming back
# #If customers bought in Month X, how many of those same customers also bought again in Month X+1?”
# # Retention (Jan → Feb) =
# # Customers who returned in Feb
# # ------------------------------
# # Total customers in Jan

import pandas as pd

Online_Sales['Transaction_Date'] = pd.to_datetime(Online_Sales['Transaction_Date'])
Online_Sales['Month'] = Online_Sales['Transaction_Date'].dt.to_period('M')

# Monthly unique customers (for retention)
monthly_customers = (
    Online_Sales
    .groupby('Month')['CustomerID']
    .unique()          # reduce duplicates first
    .apply(set)        # convert to set for intersection
)

# Month-over-month retention calculation
retention_records = []
months = monthly_customers.index

for i in range(1, len(months)):
    prev_cust = monthly_customers.iloc[i - 1]
    curr_cust = monthly_customers.iloc[i]

    retention_rate = len(prev_cust & curr_cust) / len(prev_cust)

    retention_records.append({
        'Month': months[i],
        'Retention_Rate': retention_rate
    })

retention_df = pd.DataFrame(retention_records)

# Identify high- & low-retention months
threshold = retention_df['Retention_Rate'].quantile(0.75)
```

```
avg_retention = retention_df['Retention_Rate'].mean()

high_retention_months = retention_df.loc[
    retention_df['Retention_Rate'] >= threshold, 'Month'
]

low_retention_months = retention_df.loc[
    retention_df['Retention_Rate'] < avg_retention, 'Month'
]

# Filter sales for behavior analysis
high_retention_sales = Online_Sales[
    Online_Sales['Month'].isin(high_retention_months)
].copy()

low_retention_sales = Online_Sales[
    Online_Sales['Month'].isin(low_retention_months)
].copy()

# Behavioral analysis
# ---- Product category patterns ----
high_retention_category = high_retention_sales['Product_Category'].value_counts()
low_retention_category = low_retention_sales['Product_Category'].value_counts()

high_retention_category_pct = (
    high_retention_category / high_retention_category.sum()
)

low_retention_category_pct = (
    low_retention_category / low_retention_category.sum()
)

# ---- Coupon usage patterns ----
high_retention_coupon = high_retention_sales['Coupon_Status'].value_counts(normalize=True)
low_retention_coupon = low_retention_sales['Coupon_Status'].value_counts(normalize=True)

# ---- Transaction value analysis ----
high_retention_value_stats = high_retention_sales['Invoice_Value'].describe()
low_retention_value_stats = low_retention_sales['Invoice_Value'].describe()

# Outputs
print("Monthly Retention Rates:")
print(retention_df)

print("\nHigh Retention Months:")
print(high_retention_months)

print("\nProduct Categories (High Retention Months):")
print(high_retention_category_pct)

print("\nProduct Categories (Low Retention Months):")
print(low_retention_category_pct)

print("\nCoupon Usage (High Retention Months):")
print(high_retention_coupon)

print("\nCoupon Usage (Low Retention Months):")
print(low_retention_coupon)

print("\nTransaction Value Stats (High Retention Months):")
print(high_retention_value_stats)

print("\nTransaction Value Stats (Low Retention Months):")
print(low_retention_value_stats)
```

```
Accessories             0.006414
Nest-Canada             0.005627
Bottles                 0.004599
Gift Cards              0.003782
Housewares              0.001422
Android                 0.000454
Fun                     0.000192
Backpacks               0.000159
Google                  0.000134
More Bags               0.000103
Name: count, dtype: float64

Coupon Usage (High Retention Months):
Coupon_Status
Clicked     0.508095
Used        0.338059
Not Used    0.153846
Name: proportion, dtype: float64

Coupon Usage (Low Retention Months):
Coupon_Status
Clicked     0.508793
Used        0.338394
Not Used    0.152813
Name: proportion, dtype: float64

Transaction Value Stats (High Retention Months):
count    185604.000000
mean         72.219962
std         153.179290
min           6.608300
25%          15.808160
50%          25.702140
75%          97.630000
max        9940.208550
Name: Invoice_Value, dtype: float64

Transaction Value Stats (Low Retention Months):
count    396408.000000
mean         95.101776
std         148.619785
min           6.308000
25%          19.795380
50%          48.781760
75%         124.310000
max        8979.275000
Name: Invoice_Value, dtype: float64
```

```
#3.Compare the revenue generated by new and existing customers month-over-month. What does this trend
#and retention efforts?

Online_Sales.head()
first_purchase

# Map first purchase date directly using indexed first_purchase
#The First_Purchase_Date column is just a helper to compare each transaction's date with the customer'
Online_Sales['First_Purchase_Date'] = Online_Sales['CustomerID'].map(first_purchase['First_Purchase_Da

# Label transactions as 'New' or 'Existing'
Online_Sales['Customer_Type'] = (
    Online_Sales['Transaction_Date'] == Online_Sales['First_Purchase_Date']
).map({True: 'New', False: 'Existing'})

# drop helper column
Online_Sales.drop(columns=['First_Purchase_Date'], inplace=True)

# Aggregate revenue per month by customer type
monthly_revenue = Online_Sales.groupby(['Month', 'Customer_Type'])['Invoice_Value'].sum().reset_index(

# Pivot for plotting
monthly_revenue_pivot = monthly_revenue.pivot(
    index='Month', columns='Customer_Type', values='Invoice_Value'
).fillna(0)

#Plot
monthly_revenue_pivot.plot(
    kind='bar', stacked=True, figsize=(12,6), colormap='viridis'
)
plt.title("Monthly Revenue: New vs Existing Customers")
plt.xlabel("Month")
```

```
plt.ylabel("Revenue (Invoice Value)")
plt.xticks(rotation=45)
plt.show()

# print pivot table
print(monthly_revenue_pivot)
```




```
Customer_Type      Existing           New
Month
2019-01        9.878135e+05  3.997650e+06
2019-02        1.375168e+06  2.516800e+06
2019-03        1.520699e+06  2.915988e+06
2019-04        2.668716e+06  2.100250e+06
2019-05        1.895937e+06  1.867719e+06
2019-06        1.801127e+06  2.073067e+06
2019-07        3.274911e+06  1.266819e+06
2019-08        2.904432e+06  2.083958e+06
2019-09        2.722518e+06  1.596836e+06
2019-10        2.752988e+06  2.096798e+06
2019-11        3.530228e+06  2.175558e+06
2019-12        3.821216e+06  2.141684e+06
```

```
#4.Identify the top-performing products and analyze the factors driving their success. How can this
#insight inform inventory management and promotional strategies?

# Aggregate total revenue and quantity sold per product
product_performance = Online_Sales.groupby('Product_Description').agg(
    Total_Revenue=('Invoice_Value', 'sum'),
    Total_Quantity=('Quantity', 'sum'),
    Avg_Price=('Avg_Price', 'mean'),
    Total_Transactions=('Transaction_ID', 'count')
).reset_index()

# Sort by revenue
top_products_revenue = product_performance.sort_values(by='Total_Revenue', ascending=False).head(10)

print("Top 10 Products by Revenue:")
top_products_revenue
```

```
Top 10 Products by Revenue:
```

| | Product_Description | Total_Revenue | Total_Quantity | Avg_Price | Total_Transactions |
|---|---|---|---|---|---|
| **316** | Nest Learning Thermostat 3rd Gen-USA - Stainle... | 7.574437e+06 | 54840 | 150.981874 | 42132 |
| **312** | Nest Cam Outdoor Security Camera - USA | 6.944411e+06 | 62472 | 121.806541 | 39936 |
| **310** | Nest Cam Indoor Security Camera - USA | 5.857953e+06 | 52824 | 120.214594 | 38760 |
| **321** | Nest Protect Smoke + CO White Battery Alarm-USA | 2.369048e+06 | 32196 | 79.838692 | 16332 |
| **323** | Nest Protect Smoke + CO White Wired Alarm-USA | 2.333684e+06 | 32040 | 79.748254 | 12780 |
| **317** | Nest Learning Thermostat 3rd Gen-USA - White | 2.253723e+06 | 16416 | 149.644555 | 13068 |

```
#5.Segment customers into groups such as Premium, Gold, Silver, and Standard. What targeted strategies
#improve retention and revenue? (Use RFM segmentation techniques)
#RFM: Recency, Frequency, Monetary

# ----------------------------
# Step 1: Calculate RFM Metrics
# ----------------------------
latest_date = Online_Sales['Transaction_Date'].max()

rfm = Online_Sales.groupby('CustomerID').agg(
    Recency=('Transaction_Date', lambda x: (latest_date - x.max()).days),  # Days since last purchase
    Frequency=('Transaction_Date', 'count'),                                # Number of transactions
    Monetary=('Invoice_Value', 'sum')                                       # Total revenue
).reset_index()
print(rfm)

# ----------------------------
# Step 2: Score each RFM dimension (1-4)
# ----------------------------
rfm['R_Score'] = pd.qcut(rfm['Recency'], 4, labels=[4,3,2,1]).astype(int)    # lower recency = better
rfm['F_Score'] = pd.qcut(rfm['Frequency'], 4, labels=[1,2,3,4]).astype(int)  # higher frequency = bett
rfm['M_Score'] = pd.qcut(rfm['Monetary'], 4, labels=[1,2,3,4]).astype(int)   # higher monetary = bette

# Combine RFM score
rfm['RFM_Score'] = rfm['R_Score'].astype(str) + rfm['F_Score'].astype(str) + rfm['M_Score'].astype(str

print(rfm)
# ----------------------------
# Step 3: Segment Customers
# ----------------------------
def segment_customer(rfm_score): #rfm_score = "434"
    r, f, m = int(rfm_score[0]), int(rfm_score[1]), int(rfm_score[2])
    if r >= 3 and f >= 3 and m >= 3:
        return 'Premium'
    elif r >= 3 and f >= 2 and m >= 2:
        return 'Gold'
    elif r >= 2 and f >= 2 and m >= 2:
        return 'Silver'
    else:
        return 'Standard'

rfm['Segment'] = rfm['RFM_Score'].apply(segment_customer)
rfm
```

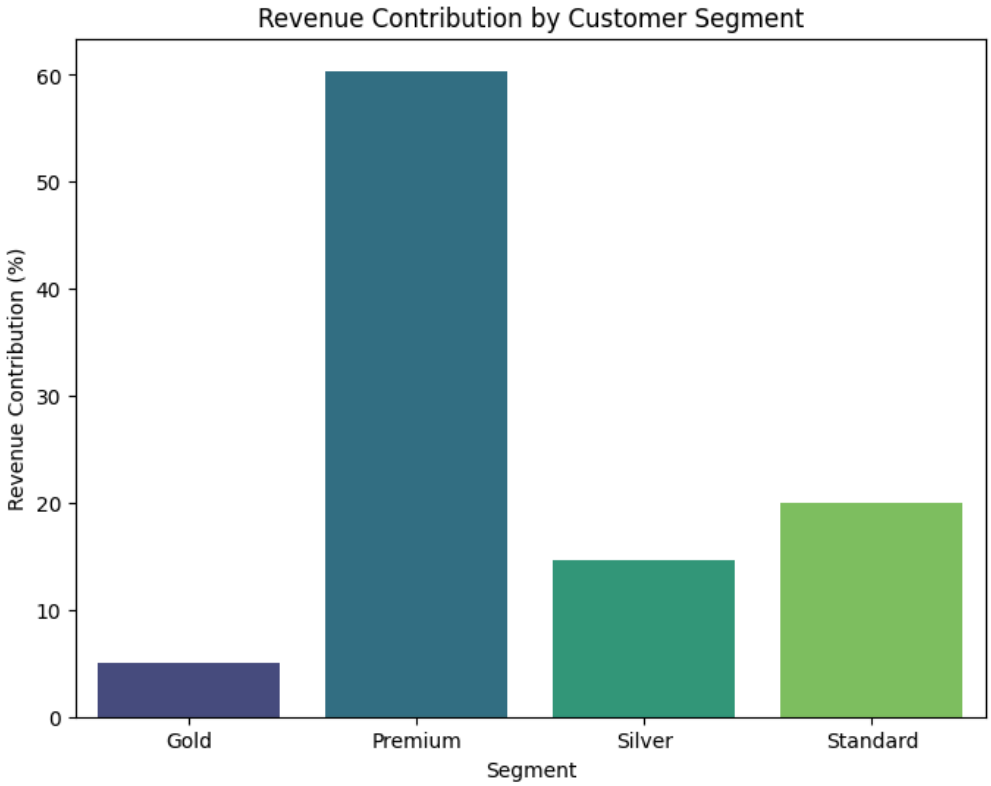
```
      CustomerID  Recency  Frequency      Monetary
0          12346      107         24    2142.60672
1          12347       59        709  160334.29440
2          12348       73        276   18164.71968
3          12350       17        204   16014.56832
4          12356      107        432   22954.48608
...          ...      ...        ...           ...
1463       18259      270         73    8780.07840
1464       18260       87        469   32184.68736
1465       18269      194         96    1766.02944
1466       18277       69         12    3218.88000
1467       18283       82       1213   75058.99488

[1468 rows x 4 columns]
      CustomerID  Recency  Frequency      Monetary  R_Score  F_Score  M_Score  \
0          12346      107         24    2142.60672        3        1        1
1          12347       59        709  160334.29440        3        4        4
2          12348       73        276   18164.71968        3        3        2
3          12350       17        204   16014.56832        4        2        2
4          12356      107        432   22954.48608        3        3        3
...          ...      ...        ...           ...      ...      ...      ...
1463       18259      270         73    8780.07840        1        1        2
1464       18260       87        469   32184.68736        3        3        3
1465       18269      194         96    1766.02944        2        1        1
1466       18277       69         12    3218.88000        3        1        1
1467       18283       82       1213   75058.99488        3        4        4

      RFM_Score
0           311
1           344
2           332
3           422
4           333
...         ...
1463        112
1464        333
1465        211
1466        311
1467        344

[1468 rows x 8 columns]
```

| | CustomerID | Recency | Frequency | Monetary | R_Score | F_Score | M_Score | RFM_Score | Segment |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 12346 | 107 | 24 | 2142.60672 | 3 | 1 | 1 | 311 | Standard |
| **1** | 12347 | 59 | 709 | 160334.29440 | 3 | 4 | 4 | 344 | Premium |
| **2** | 12348 | 73 | 276 | 18164.71968 | 3 | 3 | 2 | 332 | Gold |
| **3** | 12350 | 17 | 204 | 16014.56832 | 4 | 2 | 2 | 422 | Gold |
| **4** | 12356 | 107 | 432 | 22954.48608 | 3 | 3 | 3 | 333 | Premium |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **1463** | 18259 | 270 | 73 | 8780.07840 | 1 | 1 | 2 | 112 | Standard |

```
#6.Analyze the revenue contribution of each customer segment. How can the company focus its efforts or
#segments while nurturing lower-value segments?

# ----------------------------
# Step 4: Revenue Contribution by Segment
# ----------------------------
segment_revenue = rfm.groupby('Segment')['Monetary'].sum().reset_index()
segment_revenue['Revenue_Pct'] = segment_revenue['Monetary'] / segment_revenue['Monetary'].sum() * 100

print("Revenue Contribution by Segment:\n", segment_revenue)

# Visualization
plt.figure(figsize=(8,6))
sns.barplot(data=segment_revenue, x='Segment', y='Revenue_Pct', palette='viridis')
plt.title("Revenue Contribution by Customer Segment")
plt.ylabel("Revenue Contribution (%)")
plt.show()

# ----------------------------
# Step 5: Merge segment info back to Online_Sales for further analysis
# ----------------------------
Online_Sales = Online_Sales.merge(rfm[['CustomerID','Segment']], on='CustomerID', how='left')

# analyze average invoice, quantity, discount per segment
```

```
segment_analysis = Online_Sales.groupby('Segment').agg(
    Avg_Invoice_Value=('Invoice_Value','mean'),
    Avg_Quantity=('Quantity','mean'),
    Avg_Discount=('Discount_pct','mean'),
    Coupon_Usage=('Coupon_Status', lambda x: (x=='Used').mean()*100) # % of transactions using coupon
).reset_index()

print("\nSegment Behavior Analysis:\n", segment_analysis)

# ----------------------------
# Step 6: Strategic Recommendations
# ----------------------------
strategies = {
    'Premium': "Loyalty programs, exclusive offers, early access to new products",
    'Gold': "Upselling, cross-selling, targeted discounts",
    'Silver': "Engagement emails, personalized promotions, encourage repeat purchases",
    'Standard': "Re-engagement campaigns, introductory offers, incentives to convert"
}

for seg, strat in strategies.items():
    print(f"{seg}: {strat}")
```

```
Revenue Contribution by Segment:
     Segment      Monetary  Revenue_Pct
0       Gold  2.841166e+06     5.065472
1    Premium  3.385977e+07    60.368053
2     Silver  8.191908e+06    14.605226
3   Standard  1.119604e+07    19.961248
/tmp/ipython-input-710816969.py:14: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x

  sns.barplot(data=segment_revenue, x='Segment', y='Revenue_Pct', palette='viridis')
```




```
Segment Behavior Analysis:
     Segment  Avg_Invoice_Value  Avg_Quantity  Avg_Discount  Coupon_Usage
0       Gold          85.505191      3.598309           0.2     33.483768
1    Premium          94.765650      4.500183           0.2     34.048815
2     Silver          74.509825      4.320704           0.2     33.364536
3   Standard          86.245465      4.881756           0.2     33.787892
Premium: Loyalty programs, exclusive offers, early access to new products
Gold: Upselling, cross-selling, targeted discounts
Silver: Engagement emails, personalized promotions, encourage repeat purchases
Standard: Re-engagement campaigns, introductory offers, incentives to convert
```

```
#7.Group customers by their month of first purchase and analyze retention rates over time. Which cohor
#What strategies can be implemented to improve retention for weaker cohorts?
```

```
#A cohort is basically a group of people who share a common characteristic during a defined time peric
#Cohort Month is the month in which a customer makes their first-ever purchase, and it is used to grou

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from operator import attrgetter

print(first_purchase)

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from operator import attrgetter

# ----------------------------
# Step 0: Reset index (CustomerID is index)
# ----------------------------
first_purchase = first_purchase.reset_index()

# ----------------------------
# Step 1: create cohort month
# ----------------------------
# Cohort month derived from first purchase date
first_purchase['Cohort_Month'] = first_purchase['First_Purchase_Date'].dt.to_period('M')
Online_Sales['Transaction_Month'] = Online_Sales['Transaction_Date'].dt.to_period('M')

# ----------------------------
# Step 2: Merge cohort info into Online_Sales
# ----------------------------
Online_Sales = Online_Sales.merge(
    first_purchase[['CustomerID', 'Cohort_Month']],
    on='CustomerID',
    how='left'
)

# ----------------------------
# Step 3: Calculate cohort index (months since first purchase)
# Cohort_Index = how many months after first purchase this transaction occurred.
# .apply(attrgetter('n')) converts the Period difference into an integer.
# ----------------------------
Online_Sales['Cohort_Index'] = (
    Online_Sales['Transaction_Month'] - Online_Sales['Cohort_Month']
).apply(attrgetter('n'))

# ----------------------------
# Step 4: Count unique customers per cohort & period
# Cohort_Month : Month when customers made their first-ever purchase.
# Cohort_Index: How many months after the first purchase this transaction occurred.
# CustomerID:Actually represents number of unique customers active in that cohort at that month (from
# ----------------------------
cohort_data = (
    Online_Sales
    .groupby(['Cohort_Month', 'Cohort_Index'])['CustomerID']
    .nunique()
    .reset_index()
)

print("COHORT DATA\n",cohort_data)

cohort_counts = cohort_data.pivot(
    index='Cohort_Month',
    columns='Cohort_Index',
    values='CustomerID'
)

print("COHORT COUNTS\n",cohort_counts)
# ----------------------------
# Step 5: Calculate retention rates
# ----------------------------
cohort_sizes = cohort_counts.iloc[:, 0]   # Month 0 customers
print("COHORT SIZES\n",cohort_sizes)

#Divide every row in cohort_counts by its month 0 value (cohort_sizes).
```

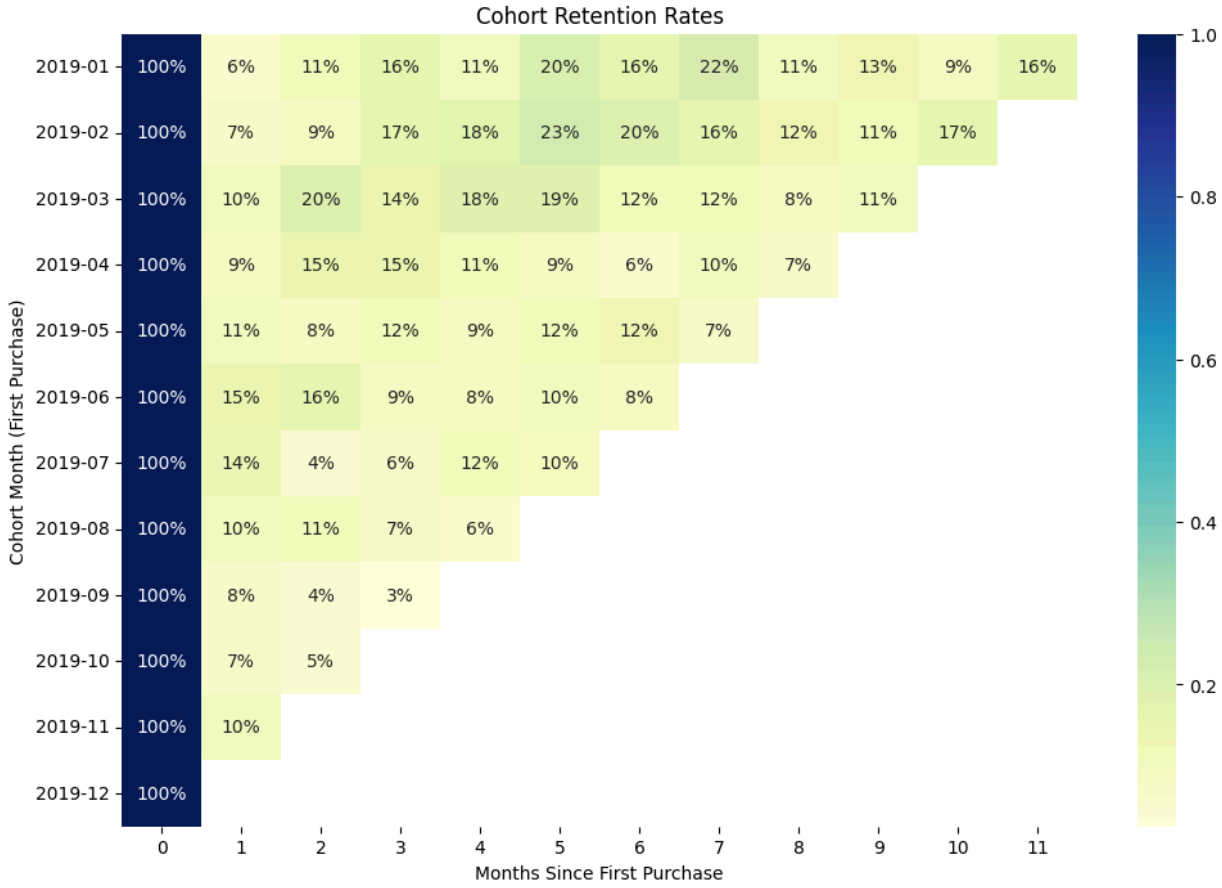
```
#Result: fraction of customers retained per month for that cohort.
retention = cohort_counts.divide(cohort_sizes, axis=0)
print("Retention Table (first 6 months):")
print(retention.iloc[:, :6])

# ----------------------------
# Step 6: Identify best & worst cohorts
# ----------------------------
avg_retention_per_cohort = retention.mean(axis=1)


#Customers who joined in December 2019 were the most loyal and kept coming back,
#while customers who joined in April 2019 were the least loyal and stopped buying quickly.
highest_retention_cohort = avg_retention_per_cohort.idxmax()
lowest_retention_cohort = avg_retention_per_cohort.idxmin()

print("\nCohort with Highest Retention:", highest_retention_cohort)
print("Cohort with Lowest Retention:", lowest_retention_cohort)

# ----------------------------
# Step 7: Visualize retention heatmap
# ----------------------------
plt.figure(figsize=(12, 8))
sns.heatmap(retention, annot=True, fmt='.0%', cmap='YlGnBu')
plt.title("Cohort Retention Rates")
plt.ylabel("Cohort Month (First Purchase)")
plt.xlabel("Months Since First Purchase")
plt.show()
```

```
            First_Purchase_Date Acquisition_Month
CustomerID
```

```
#8.Analyze the lifetime value of customers acquired in different months. How can this insight inform a
#LTV = total revenue a customer generates for your business over their entire relationship.

#LTV per cohort= Number of customers in that cohort /Total revenue from cohort
#Numerator → sum of all purchases made by customers in that cohort
#Denominator → number of customers who were first acquired in that cohort month

# ----------------------------
# Step 0: Ensure cohort info is in Online_Sales
# ----------------------------
# Already merged in your retention code
# Online_Sales has 'Cohort_Month', 'Transaction_Date', 'Invoice_Value'

# ----------------------------
# Step 1: Calculate total revenue per cohort per month since first purchase
# ----------------------------
cohort_revenue_data = (
    Online_Sales
    .groupby(['Cohort_Month', 'Cohort_Index'])['Invoice_Value']
    .sum()
    .reset_index()
)

print("COHORT REVENUE DATA\n", cohort_revenue_data)

# ----------------------------
# Step 2: Pivot to get revenue per cohort
# ----------------------------
cohort_revenue = cohort_revenue_data.pivot(
    index='Cohort_Month',
    columns='Cohort_Index',
    values='Invoice_Value'
)

print("COHORT REVENUE COUNTS\n", cohort_revenue)

# ----------------------------
# Step 3: Calculate total revenue per cohort
# ----------------------------
total_revenue_per_cohort = cohort_revenue.sum(axis=1)
print("TOTAL REVENUE PER COHORT\n", total_revenue_per_cohort)

# ----------------------------
# Step 4: Get cohort sizes (Month 0 customers)
# ----------------------------
cohort_sizes = cohort_counts.iloc[:, 0]
print("COHORT SIZES\n", cohort_sizes)

# ----------------------------
# Step 5: Calculate LTV per cohort
# ----------------------------
ltv_per_cohort = (total_revenue_per_cohort / cohort_sizes).reset_index()
ltv_per_cohort.columns = ['Cohort_Month', 'LTV']

print("\nLTV per Cohort:\n", ltv_per_cohort)

# ----------------------------
# Step 6: Visualize LTV
# ----------------------------
plt.figure(figsize=(12,6))
sns.barplot(data=ltv_per_cohort, x='Cohort_Month', y='LTV', palette='viridis')
plt.title("Customer Lifetime Value (LTV) per Cohort")
plt.ylabel("Average LTV per Customer")
plt.xlabel("Cohort Month (First Purchase)")
plt.xticks(rotation=45)
plt.show()
```

```
Cohort_Index    0         1         2         3         4         5
Cohort_Month
2019-01       1.0  0.060465  0.111628  0.158140  0.106977  0.204651
2019-02       1.0  0.072917  0.093750  0.166667  0.177083  0.229167
2019-03       1.0  0.101695  0.197740  0.141243  0.180791  0.186441
2019-04       1.0  0.085890  0.147239  0.147239  0.110429  0.092025
```

```
2019-05       1.0  0.107143  0.080357  0.116071  0.089286  0.116071
2019-06       1.0  0.145985  0.160584  0.087591  0.080292  0.102190
2019-07       1.0  0.138298  0.042553  0.063830  0.117021  0.095745
2019-08       1.0  0.103704  0.111111  0.074074  0.059259       NaN
2019-09       1.0  0.076923  0.038462  0.025641       NaN       NaN
2019-10       1.0  0.068966  0.045977       NaN       NaN       NaN
2019-11       1.0  0.102941       NaN       NaN       NaN       NaN
2019-12       1.0       NaN       NaN       NaN       NaN       NaN

Cohort with Highest Retention: 2019-12
Cohort with Lowest Retention: 2019-04
```



```
COHORT REVENUE DATA
    Cohort_Month  Cohort_Index  Invoice_Value
0       2019-01             0   4.985464e+06
1       2019-01             1   4.911643e+05
2       2019-01             2   6.177777e+05
3       2019-01             3   1.315336e+06
4       2019-01             4   3.725911e+05
..          ...           ...            ...
73      2019-10             1   1.095309e+05
74      2019-10             2   3.232674e+04
75      2019-11             0   2.449000e+06
76      2019-11             1   5.333724e+04
77      2019-12             0   2.824557e+06

[78 rows x 3 columns]
COHORT REVENUE COUNTS
 Cohort_Index            0             1             2             3   \
Cohort_Month
2019-01       4.985464e+06  491164.34016  617777.66784  1.315336e+06
2019-02       3.400804e+06  106431.52032  154193.86656  2.685501e+05
2019-03       3.712478e+06  477177.85152  435546.15552  3.619878e+05
2019-04       2.822259e+06  282007.30944  198056.74848  3.591660e+05
2019-05       2.404961e+06   80946.89472  151956.03456  1.868485e+05
2019-06       2.306775e+06  135771.46080  163291.90176  1.915082e+05
2019-07       1.802855e+06  175061.59008   83277.07872  1.675244e+05
2019-08       2.393648e+06  121350.12384  127959.02784  3.218241e+05
2019-09       1.771458e+06   22302.42144   28908.86880  7.517858e+03
2019-10       2.599189e+06  109530.94080   32326.74240           NaN
2019-11       2.449000e+06   53337.23808           NaN           NaN
2019-12       2.824557e+06           NaN           NaN           NaN

Cohort_Index            4             5             6             7   \
Cohort_Month
2019-01       372591.10656  639599.23968  1.009282e+06  631723.66080
2019-02       286827.33216  517752.80640  2.962464e+05  534964.42464
2019-03       564947.09952  830418.61344  5.709523e+05  462298.49568
2019-04       311151.52128  352971.43872  1.835218e+05  628871.35584
2019-05       199962.10272  433206.11616  4.262244e+05  130424.81664
2019-06       122531.48160  420164.39136  1.767022e+05           NaN
2019-07       264992.24736  327335.74272           NaN           NaN
2019-08       187536.76224           NaN           NaN           NaN
2019-09                NaN           NaN           NaN           NaN
2019-10                NaN           NaN           NaN           NaN
2019-11                NaN           NaN           NaN           NaN
2019-12                NaN           NaN           NaN           NaN

Cohort_Index            8             9            10            11
Cohort_Month
2019-01       492911.08032  535567.02720  452854.90080  1.059289e+06
2019-02       195686.67360  323503.89120  537346.95936           NaN
2019-03       279911.56416  417564.51648           NaN           NaN
2019-04       208961.27232           NaN           NaN           NaN
2019-05                NaN           NaN           NaN           NaN
2019-06                NaN           NaN           NaN           NaN
2019-07                NaN           NaN           NaN           NaN
2019-08                NaN           NaN           NaN           NaN
2019-09                NaN           NaN           NaN           NaN
2019-10                NaN           NaN           NaN           NaN
```

Start coding or generate with AI.

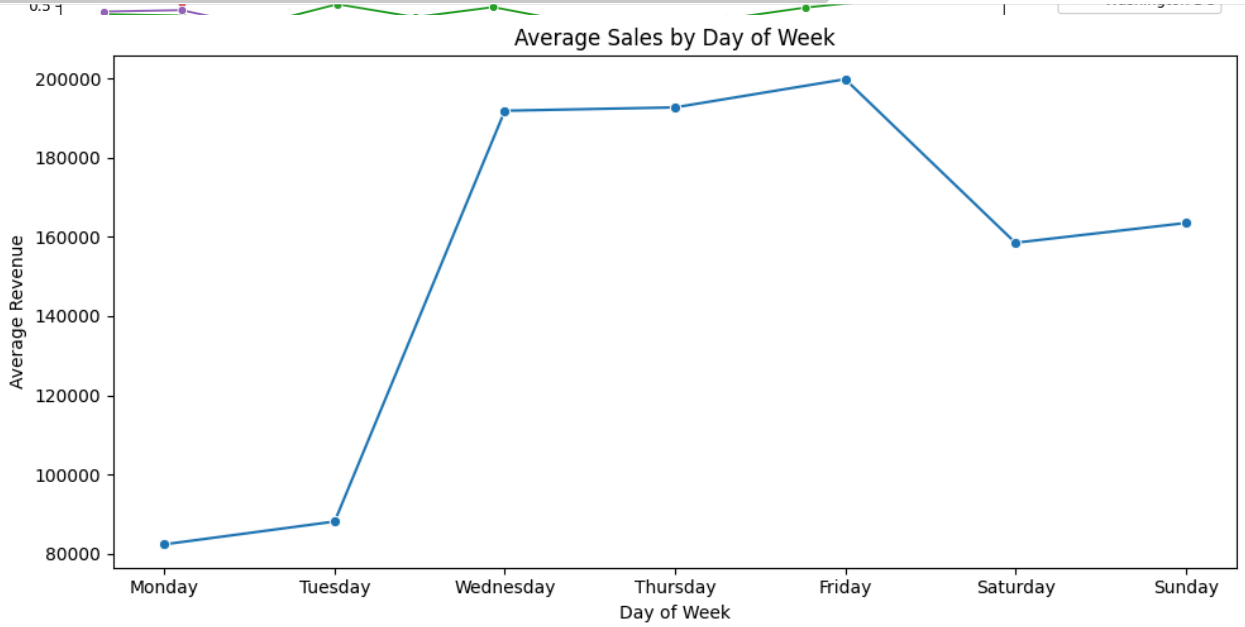
TOTAL REVENUE PER COHORT

```
#9.Identify seasonal trends in sales by category and location. How can the company prepare for peak

print(CustomerData.head())
print(Online_Sales.head())

# ----------------------------
# Step 1: Merge Customer Location into Sales Data
# ----------------------------
Online_Sales = Online_Sales.merge(
    CustomerData[['CustomerID', 'Location']],
    on='CustomerID',
    how='left'
)

print("Missing Location Count:", Online_Sales['Location'].isna().sum())

# ----------------------------
# Step 2: Create Year-Month Column
```

```
# ----------------------------
Online_Sales['YearMonth'] = Online_Sales['Transaction_Date'].dt.to_period('M')
Online_Sales['YearMonth_str'] = Online_Sales['YearMonth'].astype(str)

# ----------------------------
# Step 3: Aggregate Sales by Product Category & Month
# ----------------------------
sales_by_category = (
    Online_Sales
    .groupby(['YearMonth_str', 'Product_Category'])['Invoice_Value']
    .sum()
    .reset_index()
    .sort_values('YearMonth_str')
)

# ----------------------------
# Step 4: Aggregate Sales by Location & Month
# ----------------------------
sales_by_location = (
    Online_Sales
    .groupby(['YearMonth_str', 'Location'])['Invoice_Value']
    .sum()
    .reset_index()
    .sort_values('YearMonth_str')
)

print("Sales by Location:\n", sales_by_location)

# ----------------------------
# Step 5: Line Plot - Sales Trend by Product Category
# ----------------------------
plt.figure(figsize=(14,3))
sns.lineplot(
    data=sales_by_category,
    x='YearMonth_str',
    y='Invoice_Value',
    hue='Product_Category',
    marker='o'
)
plt.title("Monthly Sales Trend by Product Category")
plt.ylabel("Revenue")
plt.xlabel("Month")
plt.xticks(rotation=45)
plt.legend(title='Product Category', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

# ----------------------------
# Step 6: Line Plot - Sales Trend by Location
# ----------------------------
plt.figure(figsize=(12,3))
sns.lineplot(
    data=sales_by_location,
    x='YearMonth_str',
    y='Invoice_Value',
    hue='Location',
    marker='o'
)
plt.title("Monthly Sales Trend by Location")
plt.ylabel("Revenue")
plt.xlabel("Month")
plt.xticks(rotation=45)
plt.legend(title='Location', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()


print("Final Online Sales Columns:\n", Online_Sales.columns)
```

```
  CustomerID Gender    Location  Tenure_Months
0      17850      M     Chicago             12
1      13047      M  California             43
2      12583      M     Chicago             33
3      13748      F  California             30
4      15100      M  California             49
  CustomerID Transaction_ID Transaction_Date     Product_SKU  \
0      17850          16679       2019-01-01  GGOENEBJ079499
1      17850          16679       2019-01-01  GGOENEBJ079499
2      17850          16679       2019-01-01  GGOENEBJ079499
3      17850          16679       2019-01-01  GGOENEBJ079499
4      17850          16679       2019-01-01  GGOENEBJ079499

                              Product_Description Product_Category  \
0  Nest Learning Thermostat 3rd Gen-USA - Stainle...         Nest-USA
1  Nest Learning Thermostat 3rd Gen-USA - Stainle...         Nest-USA
2  Nest Learning Thermostat 3rd Gen-USA - Stainle...         Nest-USA
3  Nest Learning Thermostat 3rd Gen-USA - Stainle...         Nest-USA
4  Nest Learning Thermostat 3rd Gen-USA - Stainle...         Nest-USA

   Quantity  Avg_Price  Delivery_Charges Coupon_Status  ...    Month  \
0         1     153.71               6.5          Used  ...  2019-01
1         1     153.71               6.5          Used  ...  2019-01
2         1     153.71               6.5          Used  ...  2019-01
3         1     153.71               6.5          Used  ...  2019-01
4         1     153.71               6.5          Used  ...  2019-01

   Customer_Type   Segment Transaction_Month Cohort_Month Cohort_Index  \
0            New  Standard           2019-01      2019-01            0
1            New  Standard           2019-01      2019-01            0
2            New  Standard           2019-01      2019-01            0
3            New  Standard           2019-01      2019-01            0
4            New  Standard           2019-01      2019-01            0

  Location_x YearMonth  YearMonth_str Location_y
0    Chicago   2019-01        2019-01    Chicago
1    Chicago   2019-01        2019-01    Chicago
2    Chicago   2019-01        2019-01    Chicago
3    Chicago   2019-01        2019-01    Chicago
4    Chicago   2019-01        2019-01    Chicago

[5 rows x 23 columns]
Missing Location Count: 0
/tmp/ipython-input-3900501518.py:39: FutureWarning: The default of observed=False is deprecated and
  .groupby(['YearMonth_str', 'Location'])['Invoice_Value']
Sales by Location:
    YearMonth_str       Location  Invoice_Value
0         2019-01     California   1.889301e+06
1         2019-01        Chicago   1.322199e+06
2         2019-01     New Jersey   3.806718e+05
3         2019-01       New York   9.768314e+05
4         2019-01  Washington DC   4.164602e+05
5         2019-02     California   1.133127e+06
6         2019-02        Chicago   1.419167e+06
7         2019-02     New Jersey   3.622584e+05
8         2019-02       New York   5.415799e+05
9         2019-02  Washington DC   4.358358e+05
14        2019-03  Washington DC   2.114263e+05
12        2019-03     New Jersey   2.491875e+05
13        2019-03       New York   1.213223e+06
10        2019-03     California   1.254114e+06
11        2019-03        Chicago   1.508737e+06
15        2019-04     California   1.459306e+06
16        2019-04        Chicago   1.780635e+06
17        2019-04     New Jersey   5.147529e+05
18        2019-04       New York   7.855051e+05
19        2019-04  Washington DC   2.287674e+05
20        2019-05     California   1.067940e+06
21        2019-05        Chicago   1.343578e+06
22        2019-05     New Jersey   3.419054e+05
23        2019-05       New York   9.359703e+05
24        2019-05  Washington DC   7.426222e+04
28        2019-06       New York   8.278583e+05
27        2019-06     New Jersey   4.754285e+05
29        2019-06  Washington DC   1.492530e+05
25        2019-06     California   1.203162e+06
26        2019-06        Chicago   1.218491e+06
30        2019-07     California   1.454948e+06
31        2019-07        Chicago   1.588920e+06
```

```
#10.Analyze daily sales trends to identify high-performing and low-performing days. What strategies

# ----------------------------
# Step 1: Aggregate daily sales
# ----------------------------
daily_sales = (
    Online_Sales
    .groupby('Transaction_Date')['Invoice_Value']
    .sum()
    .reset_index()
)

# ----------------------------
# Step 2: Aggregate by day of week to find trends
# ----------------------------
daily_sales['DayOfWeek'] = daily_sales['Transaction_Date'].dt.day_name()
sales_by_day = (
    daily_sales
    .groupby('DayOfWeek')['Invoice_Value']
    .mean()  # average revenue per day of week
    .reindex(['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday'])  # order days
    .reset_index()
)

# ----------------------------
# Step 3: Visualize daily sales trend
# ----------------------------
plt.figure(figsize=(10,5))
sns.lineplot(data=sales_by_day, x='DayOfWeek', y='Invoice_Value', marker='o')
plt.title("Average Sales by Day of Week")
plt.ylabel("Average Revenue")
plt.xlabel("Day of Week")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# ----------------------------
# Step 4: Identify high-performing and low-performing days
# ----------------------------
high_perf_day = sales_by_day.loc[sales_by_day['Invoice_Value'].idxmax(), 'DayOfWeek']
low_perf_day = sales_by_day.loc[sales_by_day['Invoice_Value'].idxmin(), 'DayOfWeek']

print(f"High-performing day: {high_perf_day}")
print(f"Low-performing day: {low_perf_day}")
```




```
High-performing day: Friday
Low-performing day: Monday
```

```
#End
```